IND320 Project Part 1

Reservoir Data Analysis

This notebook contains the data analysis for Part 1 of the IND320 project. The dataset contains historical data on Norwegian reservoir levels and energy storage capacity.

The analysis includes data preparation, exploration and visualization of the reservoir data.

1. Importing and loading the data

First, Pandas is imported and the `reservoirs.csv` dataset is loaded into a DataFrame. The first five rows are displayed to get an initial overview of the dataset and its variables.

In [ ]:
```
import pandas as pd
import matplotlib.pyplot as plt

#load the reservoir dataset from the local csv file
df = pd.read_csv("reservoirs.csv")

#display the first rows to inspect the original data
df.head()
```

Out[ ]:

	dato_Id	omrType	omrnr	iso_aar	iso_uke	fyllingsgrad	kapasitet_TWh	fylling_TV
0	1995-09-03	EL	4	1995	35	0.944721	21.079208	19.9139
1	2020-11-15	EL	1	2020	46	0.974361	6.003264	5.8493
2	2011-08-28	EL	4	2011	34	0.786499	21.079208	16.5787
3	1998-07-05	EL	3	1998	27	0.793969	8.920932	7.0829
4	2011-03-27	EL	4	2011	12	0.300820	21.079208	6.3410

2. Renaming the columns

The original dataset contains Norwegian column names. To make the dataset easier to understand and work with, the columns are renamed using clear English names.

In [ ]:
```
#rename the original norwegian columns to clear english names
df = df.rename(columns={
    "dato_Id": "date",
    "omrType": "area_type",
    "omrnr": "area_number",
    "iso_aar": "year",
    "iso_uke": "week",
    "fyllingsgrad": "fill_level",
    "kapasitet_TWh": "capacity_TWh",
    "fylling_TWh": "stored_energy_TWh",
    "neste_Publiseringsdato": "next_publication_date",
    "fyllingsgrad_forrige_uke": "fill_level_previous_week",
    "endring_fyllingsgrad": "change_in_fill_level"
})

#check that the new column names were applied correctly
df.head()
```

Out[ ]:

	date	area_type	area_number	year	week	fill_level	capacity_TWh	stored_ene
0	1995-09-03	EL	4	1995	35	0.944721	21.079208	1
1	2020-11-15	EL	1	2020	46	0.974361	6.003264	
2	2011-08-28	EL	4	2011	34	0.786499	21.079208	1
3	1998-07-05	EL	3	1998	27	0.793969	8.920932	
4	2011-03-27	EL	4	2011	12	0.300820	21.079208	

3. Exploring the dataset

Before creating visualizations, it is useful to examine the size, structure and data types of the dataset. This provides an overview of the available variables and helps identify how the data should be handled in the following analysis.

In [ ]:
```
#show the number of rows and columns in the dataset
print("Number of rows:", df.shape[0])
print("Number of columns:", df.shape[1])

#inspect column names, data types and missing values
df.info()
```

```
Number of rows: 14877
Number of columns: 11
<class 'pandas.DataFrame'>
RangeIndex: 14877 entries, 0 to 14876
Data columns (total 11 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   date                      14877 non-null  str
 1   area_type                 14877 non-null  str
 2   area_number               14877 non-null  int64
 3   year                      14877 non-null  int64
 4   week                      14877 non-null  int64
 5   fill_level                14877 non-null  float64
 6   capacity_TWh              14877 non-null  float64
 7   stored_energy_TWh         14877 non-null  float64
 8   next_publication_date     14877 non-null  str
 9   fill_level_previous_week  14877 non-null  float64
 10  change_in_fill_level      14877 non-null  float64
dtypes: float64(5), int64(3), str(3)
memory usage: 1.7 MB
```

4. Displaying the data

The dataset contains 14,877 observations and 11 variables. To get a clearer overview of the actual reservoir data, a selection of relevant columns and observations is displayed below.

In [ ]:
```python
#display the first ten observations for a closer look at the data
display(df.head(10))
```

	date	area_type	area_number	year	week	fill_level	capacity_TWh	stored_energ
0	1995-09-03	EL	4	1995	35	0.944721	21.079208	19.
1	2020-11-15	EL	1	2020	46	0.974361	6.003264	5.
2	2011-08-28	EL	4	2011	34	0.786499	21.079208	16.
3	1998-07-05	EL	3	1998	27	0.793969	8.920932	7.
4	2011-03-27	EL	4	2011	12	0.300820	21.079208	6.
5	2017-11-05	EL	4	2017	44	0.794520	21.079208	16.
6	2003-03-02	EL	1	2003	9	0.195046	6.003264	1
7	2000-11-26	EL	4	2000	47	0.866371	21.079208	18.
8	2016-09-04	EL	1	2016	35	0.890967	6.003264	5.
9	2023-08-13	EL	5	2023	32	0.836849	17.390587	14.

5. Visualizing individual columns

To better understand the dataset, the relevant numerical variables are visualized separately. The data is sorted by date before plotting so that changes over time are displayed chronologically.

In [ ]:
```
#convert the date column to datetime and sort the data chronologically
df["date"] = pd.to_datetime(df["date"])
df = df.sort_values("date")

df.head()
```

Out[ ]:

	date	area_type	area_number	year	week	fill_level	capacity_TWh	stored_
3468	1995-01-08	EL	5	1995	1	0.616448	17.390587	
3361	1995-01-08	EL	4	1995	1	0.558864	21.079208	
5036	1995-01-08	EL	3	1995	1	0.602376	8.920932	
3113	1995-01-08	EL	1	1995	1	0.621467	6.003264	
7739	1995-01-08	EL	2	1995	1	0.630947	34.043590	

In [ ]:

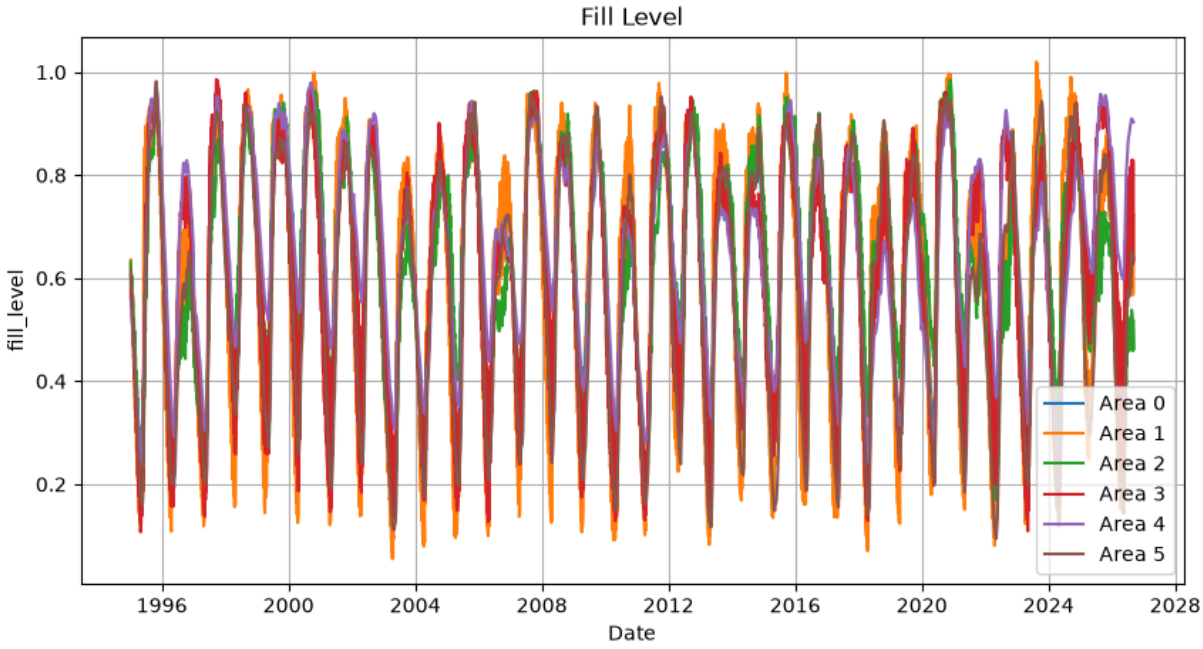
```
#select the numerical reservoir variables to visualize
numeric_columns = [
    "fill_level",
    "capacity_TWh",
    "stored_energy_TWh",
    "fill_level_previous_week",
    "change_in_fill_level"
]

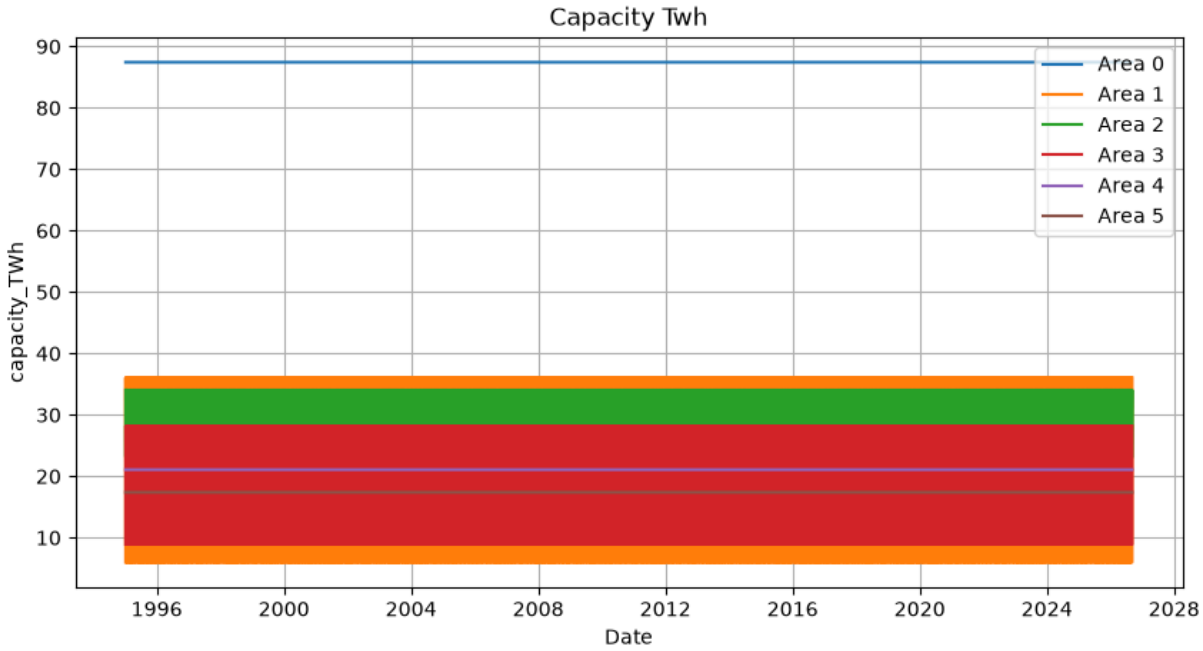
#plot each numerical variable separately for every reservoir area
for column in numeric_columns:
    plt.figure(figsize=(10, 5))

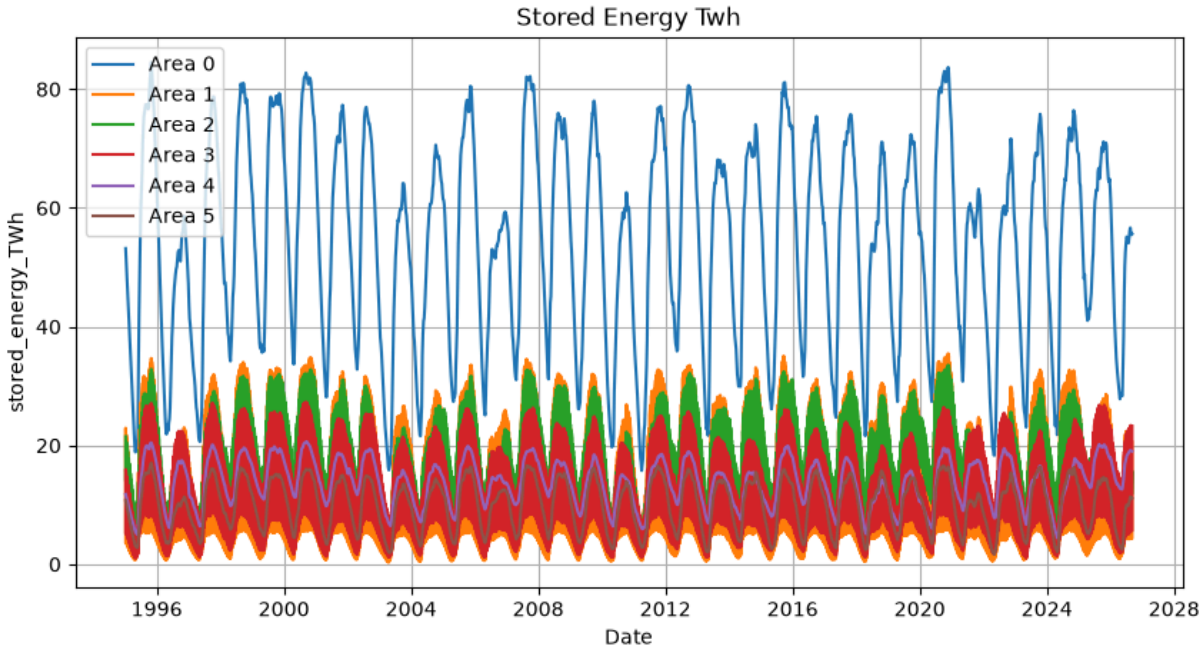
    for area in sorted(df["area_number"].unique()):
        area_data = df[df["area_number"] == area]

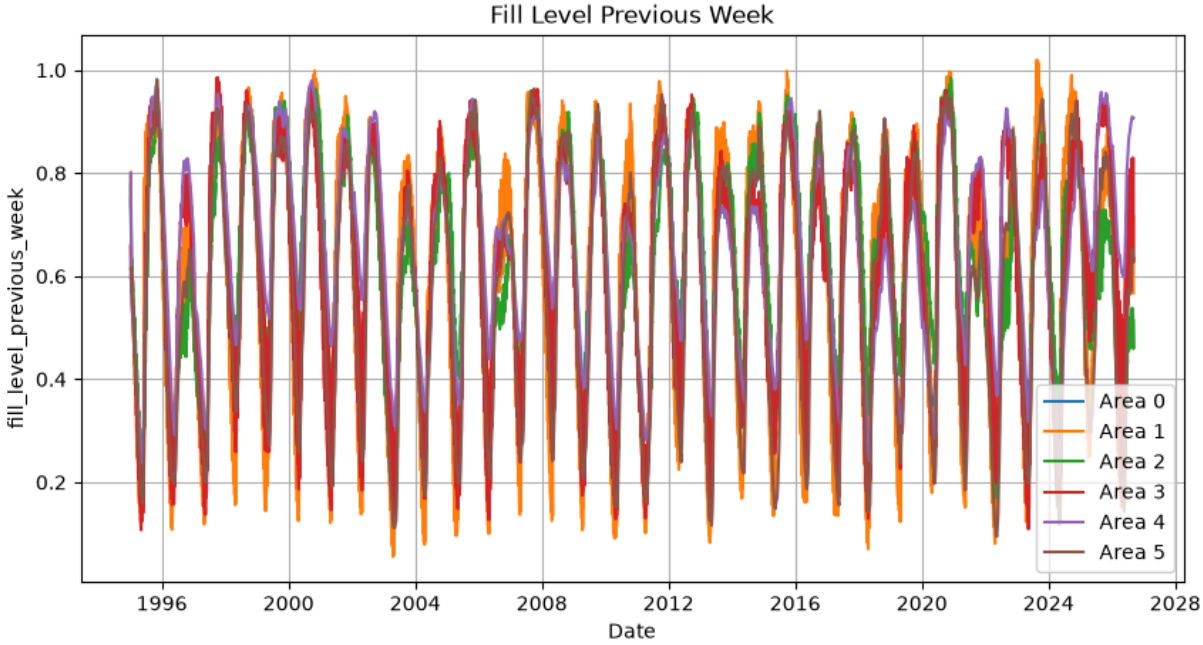
        plt.plot(
            area_data["date"],
            area_data[column],
            label=f"Area {area}"
        )

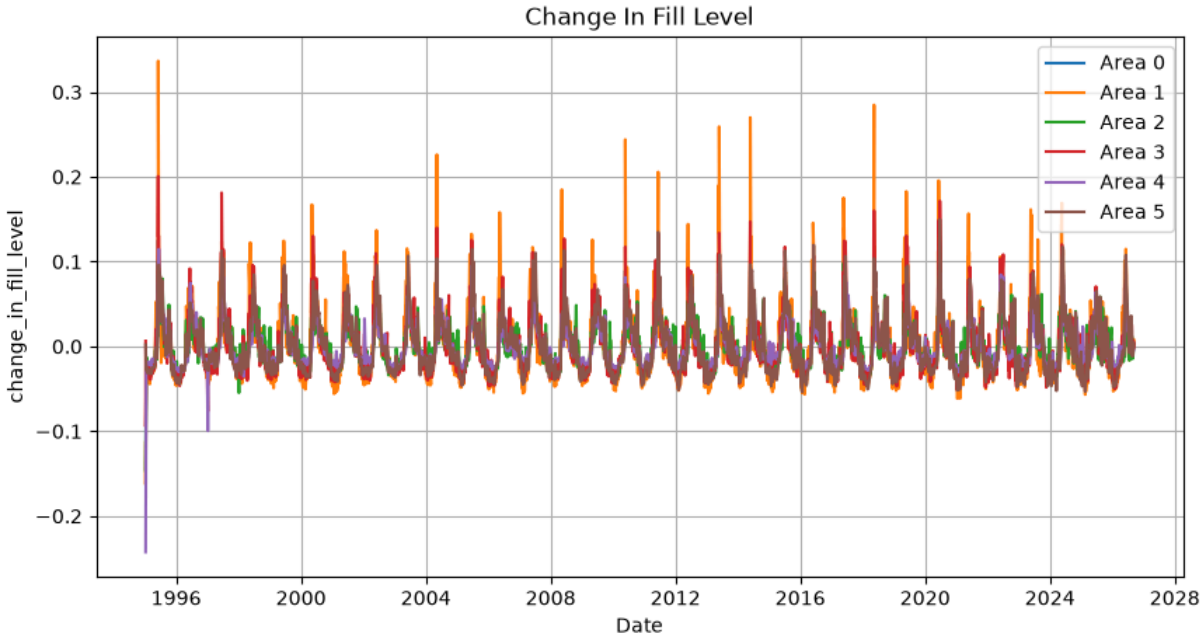
    #add labels, title, legend and grid to each plot
    plt.xlabel("Date")
    plt.ylabel(column)
    plt.title(column.replace("_", " ").title())
    plt.legend()
    plt.grid(True)
    plt.show()
```


Fill Level
fill_level
1.0
0.8
0.6
0.4
0.2
1996
2000
2004
2008
2012
2016
2020
2024
2028
Date
Area 0
Area 1
Area 2
Area 3
Area 4
Area 5


Capacity Twh
capacity_TWh
90
80
70
60
50
40
30
20
10
1996
2000
2004
2008
2012
2016
2020
2024
2028
Date
Area 0
Area 1
Area 2
Area 3
Area 4
Area 5


Stored Energy Twh
Area 0
Area 1
Area 2
Area 3
Area 4
Area 5
stored_energy_TWh
0
20
40
60
80
1996
2000
2004
2008
2012
2016
2020
2024
2028
Date


Fill Level Previous Week
fill_level_previous_week
0.2
0.4
0.6
0.8
1.0
Area 0
Area 1
Area 2
Area 3
Area 4
Area 5
1996
2000
2004
2008
2012
2016
2020
2024
2028
Date



6. Visualizing all numerical columns together

Since the numerical variables have different units and scales, they are normalized before being plotted together. This makes it possible to compare their development over time in a single figure.

In [ ]:
```
#select the numerical variables to compare
columns_to_plot = [
    "fill_level",
    "capacity_TWh",
    "stored_energy_TWh",
    "fill_level_previous_week",
    "change_in_fill_level"
]

#calculate the mean value for each date across reservoir areas
plot_data = df.groupby("date")[columns_to_plot].mean()

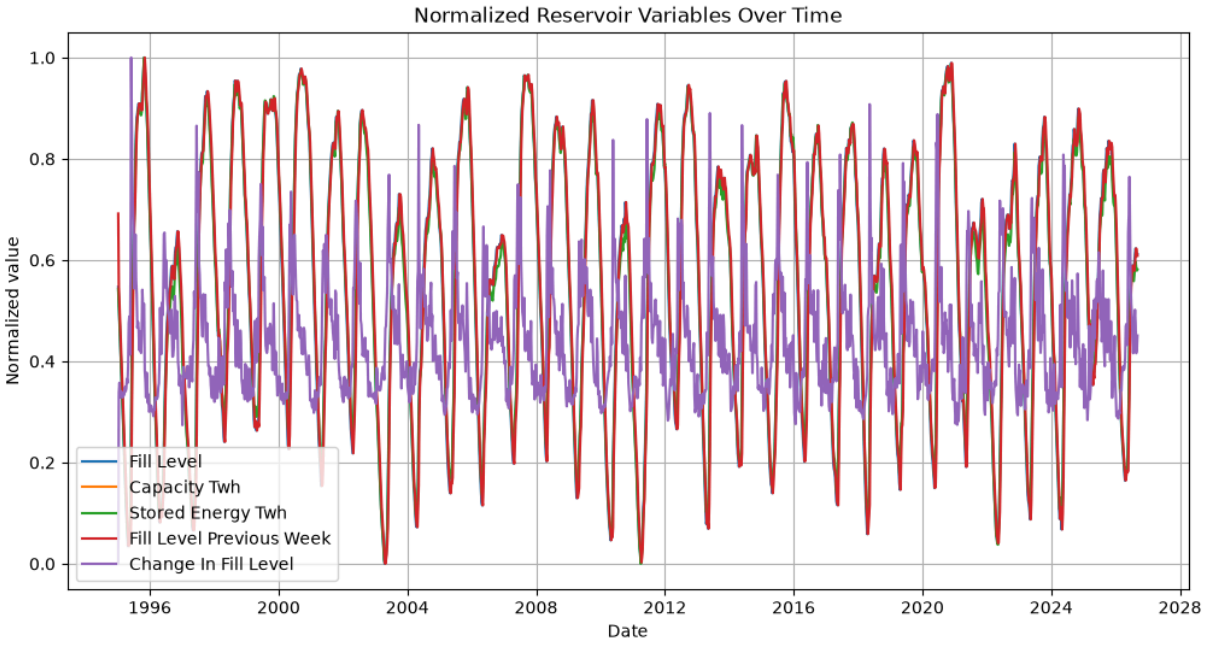
#normalize all variables to a scale from 0 to 1
normalized_df = (plot_data - plot_data.min()) / (plot_data.max() - plot_data

#plot all normalized variables together
plt.figure(figsize=(12, 6))

for column in normalized_df.columns:
    plt.plot(
        normalized_df.index,
        normalized_df[column],
        label=column.replace("_", " ").title()
    )

#add labels, title, legend and grid to the plot
plt.xlabel("Date")
```

```
plt.ylabel("Normalized value")
plt.title("Normalized Reservoir Variables Over Time")
plt.legend()
plt.grid(True)
plt.show()
```



7. Development log

The Streamlit application was developed to make the reservoir data more interactive and easier to explore. The application was divided into four separate pages, with a front page that provides navigation to the other pages. The reservoir dataset is read from the local CSV file, and caching is used so that the data does not have to be loaded again every time the user interacts with the application.

On the second page, the imported data is presented in a table. A row wise LineChartColumn is used to provide a small visualization of the first month of each data series. This gives the user a quick overview of the different variables without having to open a separate plot for each one. The third page contains the main interactive visualization. A selectbox allows the user to choose one numerical variable or display all variables together. A selection slider is also included so that the user can choose which months to display, with the first month selected by default. During development, I tested the different selections to make sure that the plot updated correctly and that the dates were displayed in chronological order.

Finally, the Streamlit project was connected to a public GitHub repository and deployed online using Streamlit Community Cloud. Working with Streamlit gave me experience with turning a local data analysis into an interactive web application. It also showed me how the structure of the data and the choices made during preprocessing in the Jupyter Notebook affect how the data can later be presented in an application.

8. AI usage

AI tools were used as support during the development of this project. ChatGPT was used to help interpret the assignment requirements, structure the Jupyter Notebook, and troubleshoot errors.

The suggested code was tested and adjusted during development. For example, the initial visualizations connected observations from different reservoir areas, which resulted in misleading plots. The plotting approach was therefore changed to separate the reservoir areas. AI was also used to suggest an approach for normalizing variables with different scales before plotting them together.

The final implementation was reviewed and tested in the local development environment.

9. Project links

GitHub repository: https://github.com/dano22222/dano22222

Streamlit application: https://dano22222-u9jpvjhwntlcuukcush85v.streamlit.app

Screencast of the assignment:
https://drive.google.com/file/d/146A_SNUnHuk8CLfHJkxZ3-moxQs2I-U_/view?usp=sharing